Predicting Hidden Roles in Multi-agent Games Using Dynamic Signed GNN

Yifan Li, Beijing Institute of Technology

Supervised by Prof. Haoran Yu (*yu-haoran.github.io*)

1. Multi-agent System with Hidden Roles

- Definition:

A multi-agent system composed of agents p with hidden roles r. Agents interact through action a.

- Examples:
  - Comment section of an online voting:

    p: users; r: voting choices; a: comment on other users
  - Jury of a case:

    p: jurors; r: verdicts; a: discussion between jurors
  - Role-playing card games:

    p: players; r: game roles; a: behavior prescribed by the rule

2. Game Resistance: Avalon

Game Resistance: Avalon is a role-playing card-based game. The game's premise involves a war between Spy and Resistance groups, and players are assigned various roles related to these groups.

The game includes 5 rounds. During each round, a player is assigned to be The Leader, who selects a certain number of players for a mission. All of the players need to vote on whether accept the team make-up. The vote(up to 5 times) repeats until a team is accepted.

Each team member needs to turn in a mission card, one for Success and the other for Fail. Only players from Resistance can select Fail card. If a Fail card is turned in, the Spies win a point. Otherwise the Resistances win a point. The team accumulating 3 points wins the game.

See https://en.wikipedia.org/wiki/The_Resistance_(game) for more detailed information.

2. Game Resistance: Avalon

Avalon is a typical multi-agent system with hidden roles.

- p: Players in the game.
- r: Unpublic game roles.
- a: Vote on the team make-up, select mission cards…

Our work is to construct graphs from game records and predict players' hidden roles by GNN.

3. Challenges

Challenge 1: Traditional methods for building signed graphs are inapplicable to our case.

Voting choices reflect players' positive or negative attitude towards the team composition, so we consider to transform the game to a ***signed graph***. A signed graph is a graph in which each edge has a positive or negative sign.

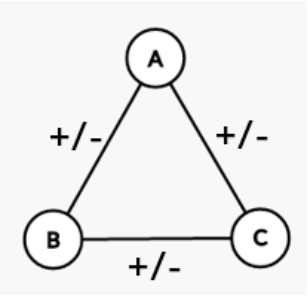


Fig 1. Signed graph

Balance theory[1] suggests "the friend of my friend is my friend" and "the foe of my foe is my foe". It considers cycles with even negative edges are *balanced* and more prevalent in real networks.

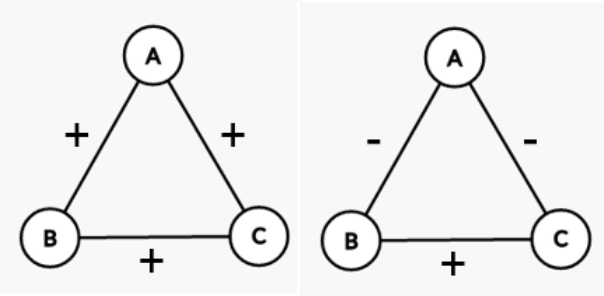


Fig 2. Balanced signed graph

[1] F. Heider, "Attitudes and cognitive organization," The Journal of psychology, vol. 21, no. 1, pp. 107–112, 1946.

Challenge 1: Traditional methods for building signed graphs are inapplicable to our case.

Most signed graph use interaction between nodes as edges (e.g. Edges in Bitcoin-OTC[2] are Bitcoin users' rating of trust levels on others).

In our case, we can build a directed 'voting graph' by **treating an 'Approve' in voting as a positive edge** from voter to every team member.

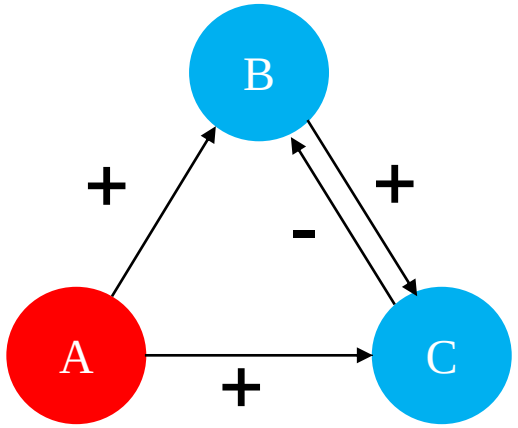
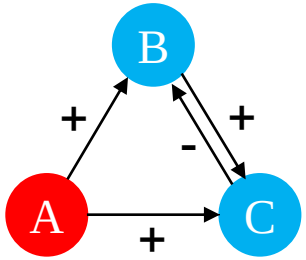


Fig 3. An example of voting graph, where B and C are team members. A, B vote for and C against the team make-up.

[2] Kumar S, Spezzano F, Subrahmanian V S, et al. Edge weight prediction in weighted signed networks[C]//2016 IEEE 16th International Conference on Data Mining (ICDM). IEEE, 2016: 221-230.

Challenge 1: Traditional methods for building signed graphs are inapplicable to our case.



Social networks: ‘A likes B’ = ‘A likes C’ + ‘C likes B’

Voting graph: **‘A supports B’ = ‘A supports C’ + ‘C supports A’ ?**

However, **Balance Theory can’t apply to voting graphs**:

- Social networks or Bitcoin platform are long-term networks and relation between users are in a dynamic equilibrium. Using the balance theory, we can suggest that A’s friendliness to B comes from the superposition of A's friendliness to C and C’s friendliness to B. This is the stable state reached by ABC after a period of interaction.
- In game Avalon, however, players know nothing but selected team members when they vote, so the voting result is **instantaneous** and **independent**. In other words, **A’s approval to B has nothing to do with C’s choice**, thus voting graph can’t be explained by Balance Theory.

Solution: Use voting results as node signs

Our solution is **using voting results as node signs** instead of edge signs. Then **we connect positive edges between nodes have same signs and vice versa**. By doing so, signed edges in graph represent whether the connected nodes have same attitude towards the team composition.

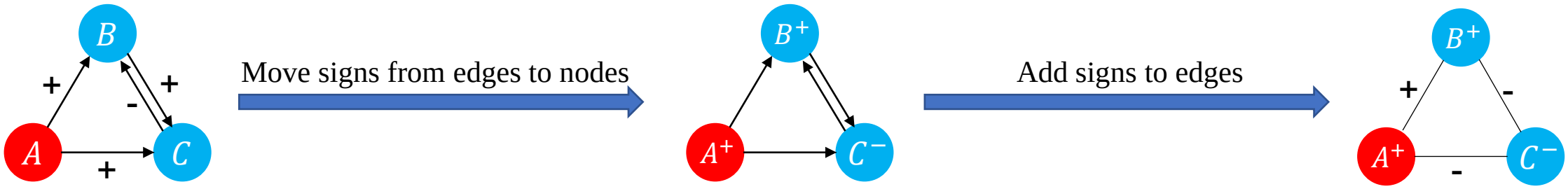


Fig 4. Change of the graph. Since the edge signs between two nodes are always the same, we make the graph undirected.

- Balance Theory now applies to the graph. ‘A disagrees with C’ = ‘A agrees with B’ + ‘B disagrees with C’.
- We can also prove that **every triangle in our graph is balanced**.

Challenge 2: Existing GNN models can't handle our graphs.

Our graphs have 3 attributes: *signed*, *heterogeneous* and *dynamic*.

- Signed:

  Our graphs are signed graph with positive and negative edges.

- Heterogeneous:

  There are 3 characters in one voting: Leader who selects the team, team members who decide the mission result and other players who only vote on the team.

- Dynamic:

  Our graphs are grouped by games. Each game has up to 5 rounds and each round has up to 5 voting. Thus there are at most 25 different graphs to describe a game.

To the best of our knowledge, **no network architecture can handle graphs with all these attributes.** Therefore, we can't simply apply existing signed/heterogeneous/dynamic graph models to our graphs.

Solution: We propose a new network architecture based on SGCN[3].

SGCN defined *balanced/unbalanced path* (*B(l)*/*U(l)*, *l* means path length) which includes even/odd number of negative edges. For each node, SGCN keeps a representation of both 'friends' and 'foes' by aggregating neighbors' on *balanced/unbalanced* paths respectively.

The aggregate layer can be described as:

$$h_i^{B(1)} = \sigma\left(W^{B(1)}[\sum_{j\in\mathcal{N}_i^+}\frac{h_j^{(0)}}{|\mathcal{N}_i^+|}, h_i^{(0)}]\right)$$

$$h_i^{U(1)} = \sigma\left(W^{U(1)}[\sum_{k\in\mathcal{N}_i^-}\frac{h_k^{(0)}}{|\mathcal{N}_i^-|}, h_i^{(0)}]\right)$$

$$h_i^{B(l)} = \sigma\left(W^{B(l)}[\sum_{j\in\mathcal{N}_i^+}\frac{h_j^{B(l-1)}}{|\mathcal{N}_i^+|}, \sum_{k\in\mathcal{N}_i^-}\frac{h_k^{U(l-1)}}{|\mathcal{N}_i^-|}, h_i^{B(l-1)}]\right)$$

$$h_i^{U(l)} = \sigma\left(W^{U(l)}[\sum_{j\in\mathcal{N}_i^+}\frac{h_j^{U(l-1)}}{|\mathcal{N}_i^+|}, \sum_{k\in\mathcal{N}_i^-}\frac{h_k^{B(l-1)}}{|\mathcal{N}_i^-|}, h_i^{U(l-1)}]\right)$$

where $N_i^{+/-}$ means friends or foe neighbors of node i.

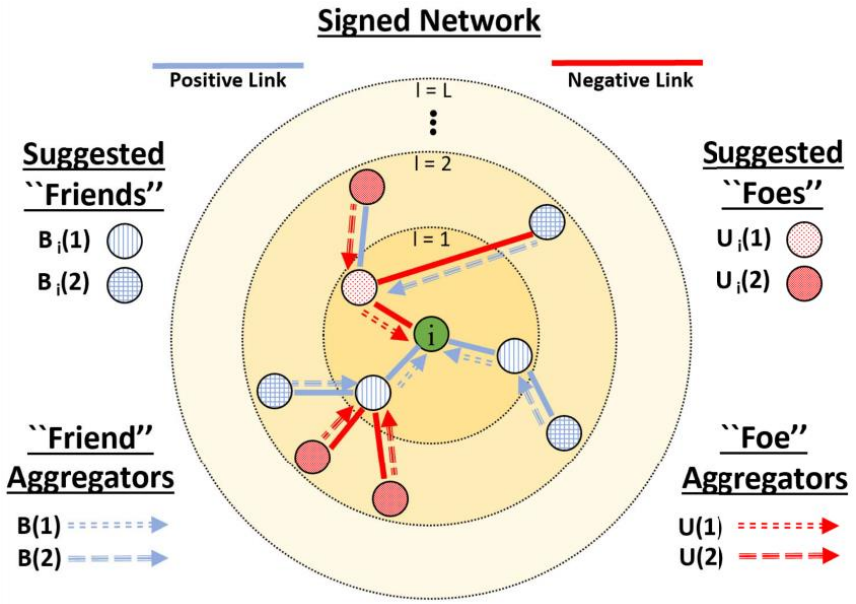


Fig 5. Illustrating how SGCN aggregates neighbor information, from[3].

[3] Derr T, Ma Y, Tang J. Signed graph convolutional networks[C]//2018 IEEE International Conference on Data Mining (ICDM). IEEE, 2018: 929-934.

Solution: We propose a new network architecture based on SGCN[3].

Heterogenization:

We combine the aggregating of SGCN with hierarchical aggregating which commonly used in heterogeneous graph.

- **Node-level aggregating**: We define 3 kinds of *meta-path* according to the character of the player at the end of the path, namely *leader*, *member* or *nonmember*. Then we respectively calculate positive and negative embedding along these meta-path.
- **Semantic-level aggregating**: We concatenate all the embeddings calculated at node-level.

Now the new aggregate layer (**HetSGCN**) can be described as (take balanced embedding as an example):

$$h_i^{B(1)} = \sigma\left(W^{B(1)}\left[\sum_{j\in\mathcal{N}_G^+(i)} \frac{h_j^{(0)}}{|\mathcal{N}_G^+(i)|}, \sum_{k\in\mathcal{N}_U^+(i)} \frac{h_k^{(0)}}{|\mathcal{N}_U^+(i)|}, \sum_{l\in\mathcal{N}_L^+(i)} \frac{h_l^{(0)}}{|\mathcal{N}_L^+(i)|}, h_i^{(0)}\right]\right)$$

$$h_i^{B(l)} = \sigma\left(W^{B(l-1)}\left[\sum_{j\in\mathcal{N}_G^+(i)} \frac{h_j^{B(l-1)}}{|\mathcal{N}_G^+(i)|}, \sum_{k\in\mathcal{N}_U^+(i)} \frac{h_k^{B(l-1)}}{|\mathcal{N}_U^+(i)|}, \sum_{l\in\mathcal{N}_L^+(i)} \frac{h_l^{B(l-1)}}{|\mathcal{N}_L^+(i)|}, \sum_{m\in\mathcal{N}_G^-(i)} \frac{h_m^{U(l-1)}}{|\mathcal{N}_G^-(i)|}, \sum_{n\in\mathcal{N}_U^-(i)} \frac{h_n^{U(l-1)}}{|\mathcal{N}_U^-(i)|}, \sum_{o\in\mathcal{N}_L^-(i)} \frac{h_o^{U(l-1)}}{|\mathcal{N}_L^-(i)|}, h_i^{B(l-1)}\right]\right)$$

Dynamization:

We refer to **wd-GCN**[4], appending a LSTM after GNNs. Each timestep in LSTM corresponds to a graph and nodes in the graph are input separately into the LSTM.

[4] Manessi F, Rozza A, Manzo M. Dynamic graph convolutional networks[J]. Pattern Recognition, 2020, 97: 107000.

4. Model

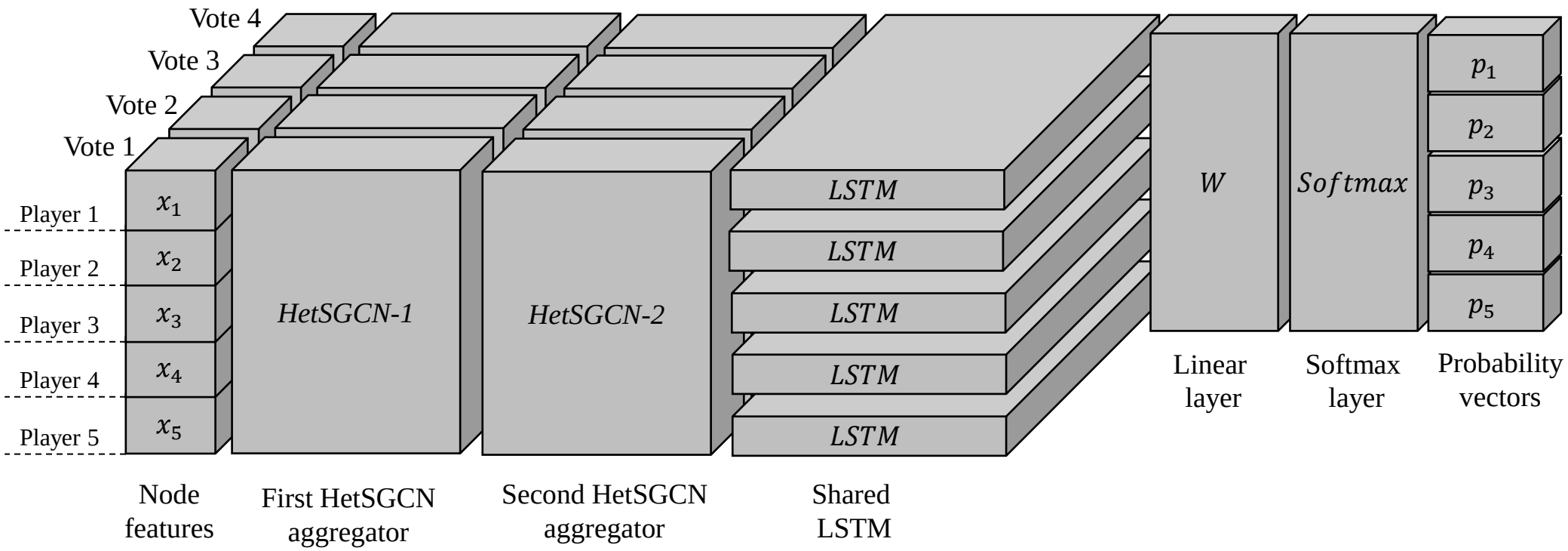


Fig 6. Our network structure. We first apply HetSGCN aggregators to collect information from 1 and 2-hop neighbors of every player. After processing all the votes, we send embeddings to an LSTM, whose parameters are shared between players. Then we take out the output from the last timestep and adjust its length to be equal with the number of hidden roles by passing it through a linear layer. Finally we calculate players' probability vectors p_i using a Softmax layer.

5. Experiment

We have accomplished the code, opening on *https://github.com/AoiDragon/Dynamic_HetSGCN*.

Now we are adjusting model parameters and comparing it with other SOTAs.

Thanks for your reading!

1120192393@bit.edu.cn